语言即操作系统

Auto

Language Made for AI OS

```
Language.new("Auto")
```

“从群众中来，到群众中去”

- Made BY AI
- Made FOR AI
- Made OF AI

2026 • 赵普明

大纲概览

1. Language × AI
2. Language as OS (LaOS)
3. 融合与突破
4. 跨平台愿景

AI 与语言的关系

探索AI与编程语言的深度关系，从"由AI创造"到"为AI设计"的语言范式变革。

Language as OS (LaOS)

突破传统三层架构，语言直接作为应用与硬件间的桥梁，实现全新操作系统范式。

融合与突破 AI + OS

AI与OS的深度融合，打破不可能三角限制，实现开发效率、程序安全与运行性能的全面突破。

愿景：Write Once, Run Everywhere

从单一OS到跨行业应用，Auto语言将扩展至汽车、机器人和通用应用开发，构建全新生态。

```
auto::outline("2024")
```

"从群众中来，到群众中去"

Intro – Auto: AI 三位一体



Made BY AI

由AI主导开发

基于100亿Tokens的AI算力，两个月内完成成传统需要数年的开发工作，由AI主导语言设计与实现。

Made FOR AI

为AI代码生成优化

专为AI生成设计的语法，双模式错误信息，注解式契约编程，让AI更容易理解和生成代码。

Made OF AI

由AI代码构成

语言本身由AI代码构成，AI可以直接理解和修改Auto代码，实现"一次生成，处处使用""的愿景。

```
auto::ai("triad")
```

Progress - Made BY AI （两个月开发奇迹）

开发周期
两个月完成传统需数年的工作

2 个月

AI算力
智谱AI MAX套餐

100亿 tokens

代码产出
高质量Rust代码实现

1300+ commits

20万 代码行数

交付成果

1个解释器
完整实现Auto语言核心功能

6个转译器
多语言/多目标平台代码生成

语言特性实现




```
auto::build(" AI miracle ")
```

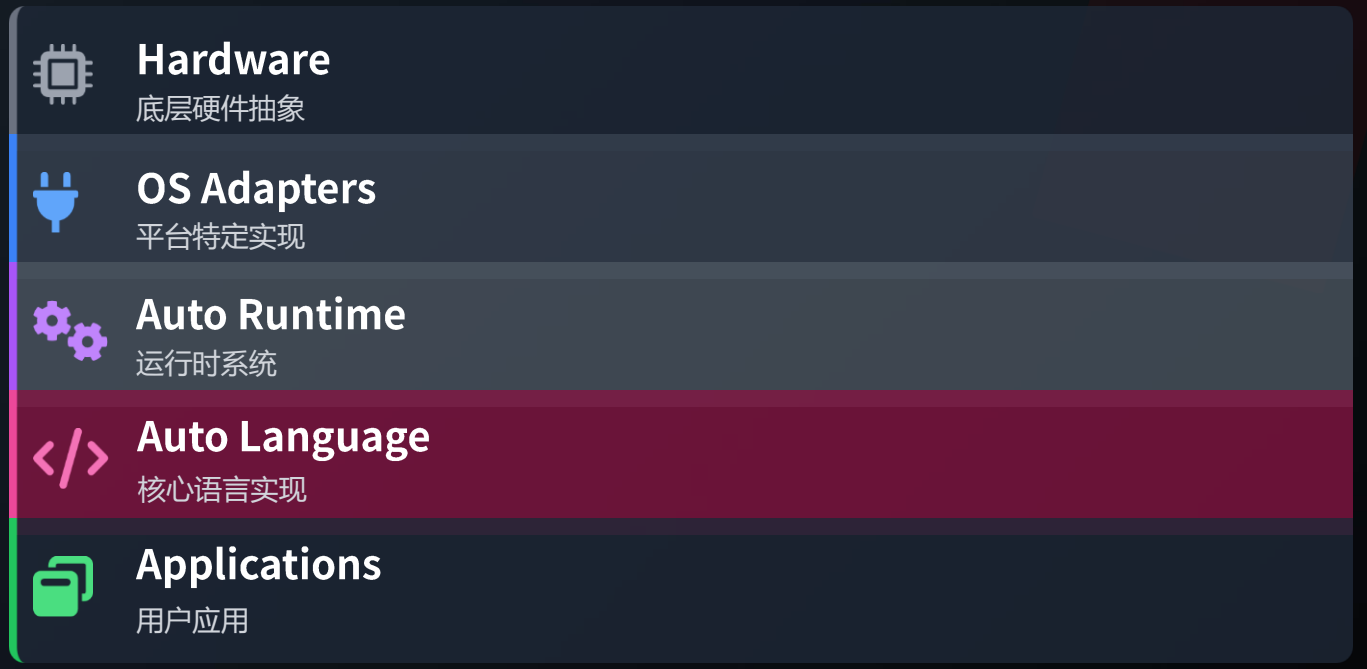
Intro - LaOS (Language as OS)

语言即操作系统

LaOS (Language as OS) 是一种新型计算架构，构，将编程语言提升为操作系统层面的抽象。

核心突破

- 突破传统"语言+运行时+OS"架构
- Auto作为应用与硬件的桥梁
- 统一系统编程模型



不可能三角的突破

⚠ 传统不可能三角

开发效率、程序安全、运行性能三者不可兼得，传统语言必须在其中两个方面做出妥协。

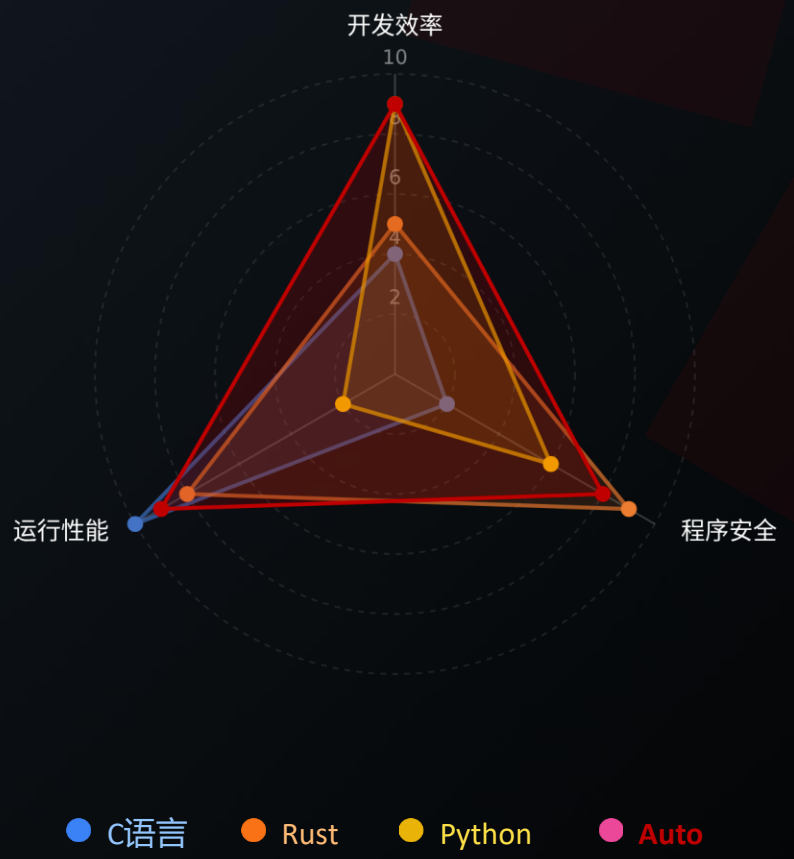
✔ Auto 的突破

Auto 打破了这一限制，在三个维度上均实现高分表现，真正实现了三者兼得。

- **Auto** ：开发效率 → **9** | 程序安全 → **8** | 运行性能 → **9**

四语言对比 (10分制)

语言	开发效率	程序安全	运行性能
C	4	2	10
Rust	5	9	8
Python	9	6	2
Auto	**9**	**8**	**9**



auto::突破不可能三角

Progress - LaOS 内核（语言特性）

AutoVM

standalone

独立运行模式，无需依赖外部解释器

comptime

编译期执行，提前计算表达式结果

embedded

前端在上位机，VM核心在MCU中

转译器矩阵

a2rs	→ Rust
a2c	→ C
a2ts	→ TypeScript
a2py	→ Python
a2kt	→ Kotlin

核心特性

CTE

编译期执行 (Compile-Time Execution)

Spec

特征系统 (Trait System)

OOP关键字 `is | has | as`

```
auto::features("Auto")
```

Progress - 内存管理

核心问题：动态内存 vs 静态内存?

动态内存

- ✓ 灵活性高，运行时分配
- ✗ 内存安全风险
- ✗ 手动管理繁琐

Auto 的统一方案

- 安全高效
- 自动回收

静态内存

- ✓ 性能高，编译时确定
- ✗ 灵活性受限
- ✗ 难处理动态数据结构

Default Storage（默认存储策略）

根据变量特性自动选择最佳内存分配策略，无需开发者干预。

Prelude Config（预置配置）

提供标准化内存管理配置，开发者可自定义特定场景行为。

```
auto::memory("management")
```

Progress - 异步并发

核心抽象： Task Msg

原生支持异步编程模型

通过~T类型表示异步计算，编译器自动处理复杂的状态机转换与协程生成。

/

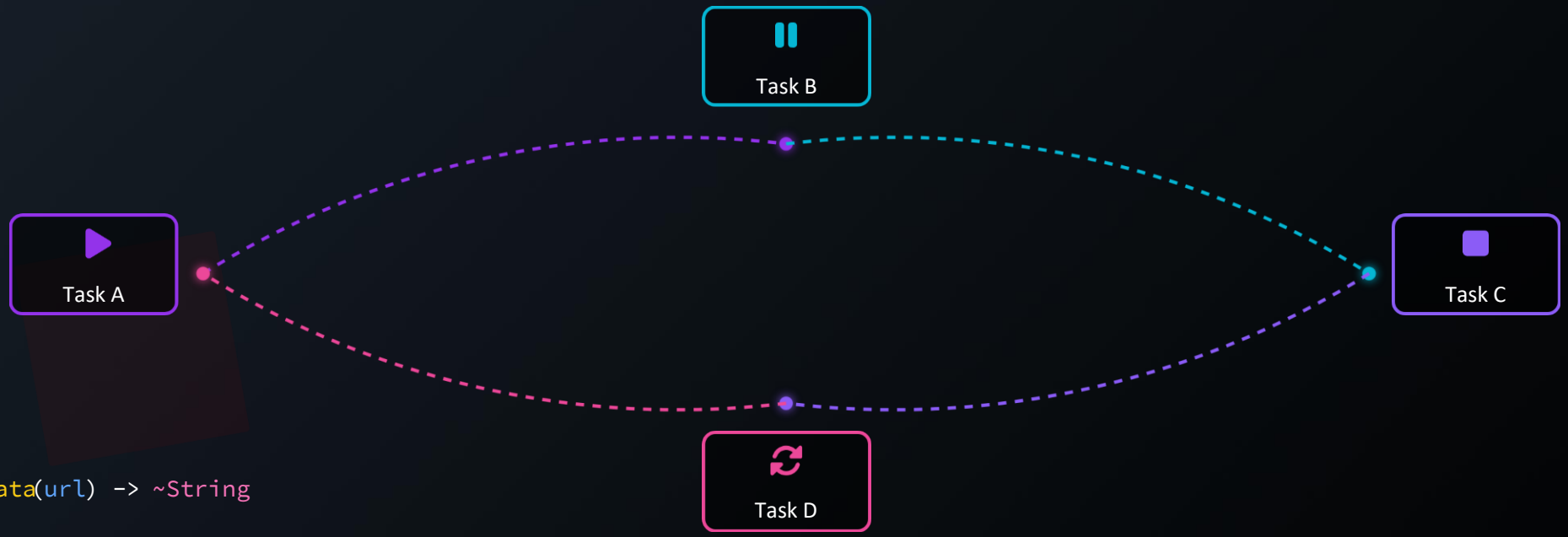
消息传递机制

Task间通过消息传递共享数据，避免共享内存带来的复杂同步问题与数据竞争。

异步类型： ~T

无数据竞争的并发

通过所有权系统和消息传递，在编译期消除数据竞争，保证程序安全性。




```
async fetch_data(url) -> ~String
```

Progress - 系统调用（标准库）

io.at
统一接口

io.vm.at
虚拟机后端

io.rs.at
Rust 后端

io.c.at
C 后端



- **一次编写，多平台运行**
  统一API，可编译至不同平台，实现"Write Once, Run Anywhere"
- **平台特定优化**
  针对不同平台特性进行优化，提供最佳性能
- **灵活后端选择**
  根据目标平台自动选择最合适的后端实现

```
io::backend("multi")
```

Progress - Shell（ASH）与 UI（Aura）

ASH (Auto Shell)

```
ash: AutoShell
```

- **基于 nushell + uutils/coreutils**
  继承强大的Posix跨平台命令实现
- **结构化管道 (Pipeline with Atom)**
  采用Atom/Batom格式（JSON的优化版）作为管道的数据格式
- **跨边界通信 (MCU <-> Windows)**
  实现不同系统间的无缝数据交换与命令执行
- **智能命令 (Smart Commands)**
  AI辅助的命令补全与自动建议功能
- **未来扩展: SafeClaw**
  计划扩展至SafeClaw，提供更安全的系统级操作

Aura (Auto UI)

```
aura: Auto UI Representation Abstract
```

多平台目标

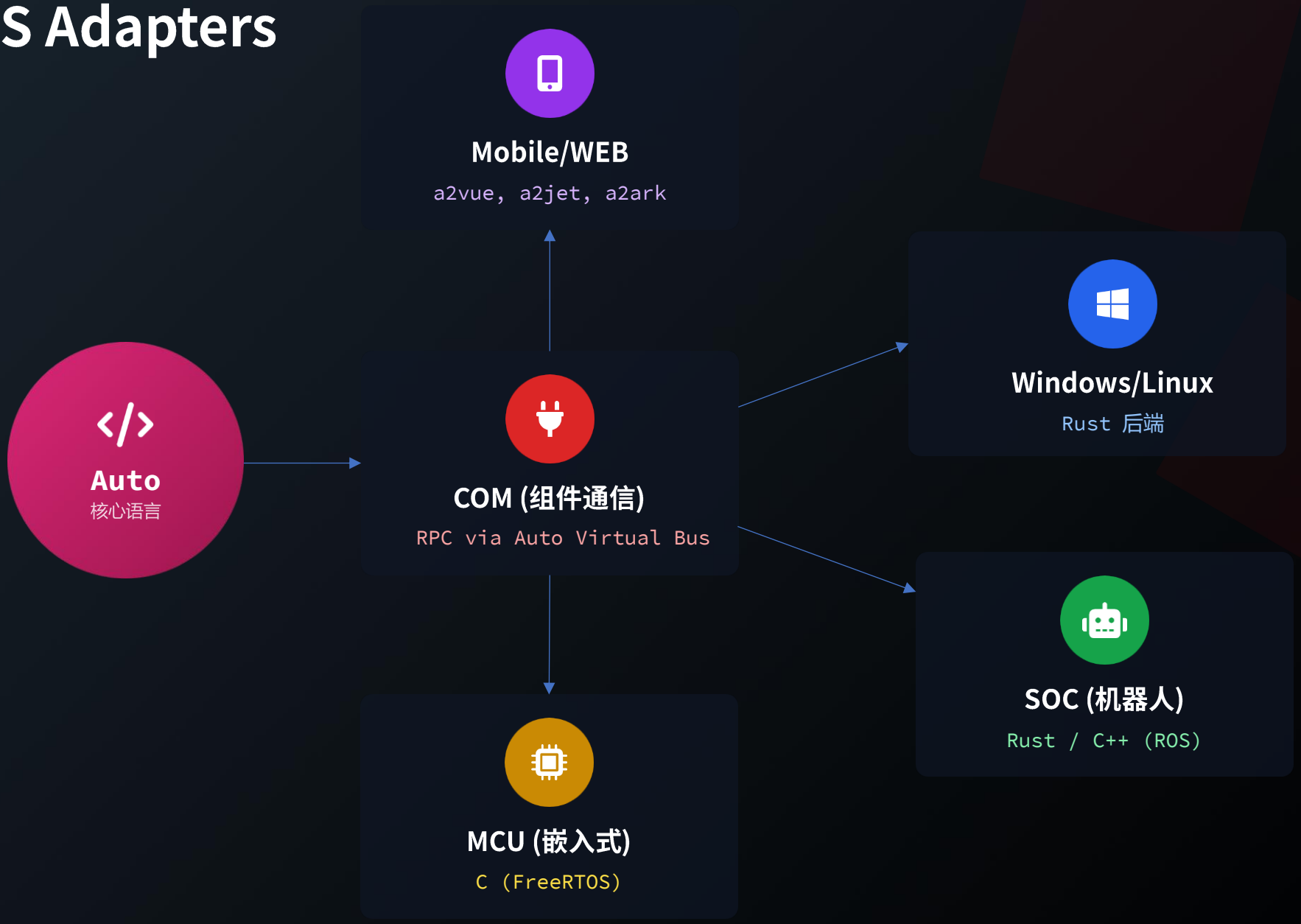
Web	Desktop	Mobile
Vue3 + shadcn-vue	Tauri / Rust (iced/gpui)	Jetpack Compose / ArkTS ArkTS

灵活后端设计

- Web → Rust HTTP Server 轻松实现前后端分离
- Desktop → Rust IPC 高效的桌面应用交互
- MCU → C + LVGL (未来) 支持嵌入式设备界面

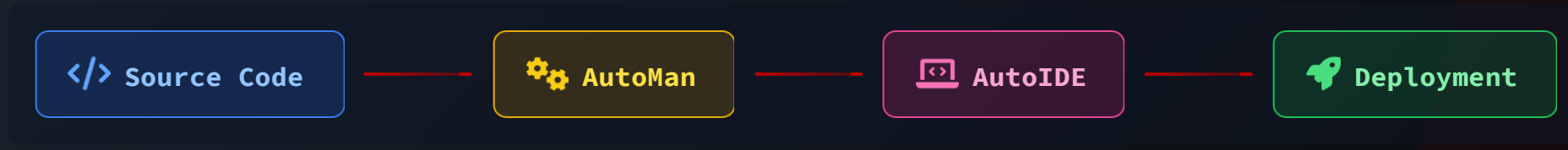
```
auto::shell("ash") auto::ui("aura")
```

Progress - OS Adapters




```
auto::adapters("多平台适配")
```

Progress - DevOp & Distribution



AutoMan 通用项目管理器

构建系统

自动化编译流程，支持多平台交叉编译，智能依赖解析与增量构建。

包管理

统一依赖管理，支持SemVer版本控制，自动解决依赖冲突，提供私有与公共仓仓库。

```
automan build --target web
```

AutoIDE 集成开发环境

Assemble them all

统一集成各类工具链，自动配置开发环境，支持多语言混合开发，提供统一调试界面。

智能辅助

AI驱动的代码补全，智能重构，实时错误分析，自动格式化与优化。

```
autoide . --no-sandbox
```

```
auto::devop("13")
```

AI - Made FOR AI



AI友好的语法设计

AI × 人类 = Auto

双模式错误信息

人类:
miette（美观的错误展示）

AI:
JSON（结构化错误信息）

注解式契约编程

```
#[in(…)]
#[out(…)]
#[invariant(…)]
fn divide(a: i32, b: i32) -> i32
```

意图注解

```
#[intent("生成REST API客户端")]
 fn create_client()
```

AI示例代码库

- 快速生成与修改
- 大规模代码库

反向转译

- 将著名开源工程反向生成Auto代码
- 理解人类注释，生成符合AI理解的注释

Claude Skill

- Auto代码生成技能
- 集成开发环境

```
auto::ai("friendly")
```

AI - Made OF AI： 自制 Agent 架构



AI - Made OF AI： AI 编译加速

Interact with AI Lang

Triton
NVIDIA

GPU通用计算

Mojo
Modular

高性能AI推理

直接转译层 - Direct transpilation

CUDA
NVIDIA

CANN
华为

ROCm
AMD

MUSA
摩尔线程

Auto直接转译到各GPU平台

AI + OS - Agent OS 生态

Cursor

初始阶段：AI驱动的代码编辑器，提供基础AI辅助编程体验。

Claude

进化阶段：通过Agent和Skill，提供更智能的开发环境。

OpenClaw

突破阶段：多Agent体系，通过tools/skills实现跨平台的AI工作环境

Agent OS

整合目标：由Agent组成的操作系统，，更安全高效的分布式智能体协作网络。

多Agent框架的其他探索

- OpenFang：更严谨的多Agent体系，初步探索Agent OS
- 三省六部制：新颖的Agent治理架构
- IronClaw：安全优先的OpenClaw，使用Rust实现

SafeClaw

- 从NanoClaw转译成Rust，再反转译成Auto
- 未来会融合IronClaw特性，增强安全性
- 集成 Auto UI，并嵌入到Auto语言的虚拟OS系统内

```
agent_os::evolution("Cursor → Claude → OpenClaw → Agent OS")
```

Beyond Single OS - 跨行业应用展望

汽车行业

整车开发

- 车载系统一体化开发
- 座舱APP: a2jet/a2ark
- SOC软件：a2rs/a2c++
- MCU底软：a2c
- 车内通讯：vbus for (ETH/CAN/LIN/..)

机器人行业

AutoOS = ROS++

- 主控芯片：a2rs/a2c++
- 执行器：a2c
- 大小脑：a2py

应用开发行业

真全栈开发

- 前后端统一开发
- 跨平台UI框架
- 后端服务与数据库集成
- 自适配跨端调用

```
auto::cross_platform()
```



感谢您的关注

我们期待与您深入交流，共同探讨Auto语言的未来



项目代码

github.com/auto-lang



文档

auto-lang.org



社区

t.me/auto_lang



联系我们

contact@auto-lang.org

Auto

```
Made BY AI , FOR AI , OF AI
```